```
      new_value = sum + 1
      print "#{new_value}  " if new_value % 250_000 == 0
      sum = new_value
      mutex.unlock
    end
  end
end
threads.each(&:join)
puts "\nsum = #{sum}"
```

produces:

```
250000  500000  750000  1000000
sum = 1000000
```

Sometimes you want to claim a lock if a mutex is currently unlocked, but you don't want to suspend the current thread if the mutex is locked. The try_lock method takes the lock if it can, but returns false if the lock is already taken. The following code illustrates a hypothetical currency converter. The ExchangeRates class caches rates from an online feed, and a background thread updates that cache once an hour. This update takes a minute or so. In the main thread, we interact with our user. However, rather than just go dead if we can't claim the mutex that protects the rate object, we use try_lock and print a status message if the update is in process.

tut_threads/mutex_3.rb

```
rate_mutex = Thread::Mutex.new
exchange_rates = ExchangeRates.new
exchange_rates.update_from_online_feed

Thread.new do
  loop do
    sleep(3600)
    rate_mutex.synchronize do
      exchange_rates.update_from_online_feed
    end
  end
end

loop do
  print "Enter currency code and amount: "
  line = gets
  if rate_mutex.try_lock
    begin
      puts(exchange_rates.convert(line))
    ensure
      puts "Ensuring unlock"
      rate_mutex.unlock
    end
  else
    puts "Sorry, rates being updated. Try again in a minute"
  end
end
```

By using ensure the unlock command is guaranteed to run even if the puts raises an exception.

If you are holding the lock on a mutex and you want to temporarily unlock it, allowing others to use it, you can call Mutex#sleep. We could use this to rewrite the previous example: